

Hyperparameter tuning with grid search and randomized search

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Distinguish trainable parameters from hyperparameters and explain why hyperparameters need a search strategy.
- Apply grid search (`GridSearchCV`) and explain why its cost grows exponentially with more hyperparameters or candidate values.
- Apply randomized search (`RandomizedSearchCV`), choose a reasonable `n_iter`, and explain why it reliably lands close to the best combination.
- Compare grid search and randomized search on a worked hospital no-show dataset and judge which one fits a given tuning problem.

2. Detailed Explanation

Trainable parameters vs. hyperparameters

Every machine learning model has two categories of parameters, and telling them apart is the foundation for everything else in this lesson.

Trainable parameters are values the training algorithm learns on its own. The algorithm takes the input variables (X) and the output variable (y) and produces the weight values that define the final prediction function.

Hyperparameters are external settings a practitioner decides *before* training starts; they are not learned from the data. Examples include the learning rate (also called eta or

alpha), the number of trees in a random forest, and the depth of a decision tree. Further examples include the minimum number of instances required per leaf, the number of layers in a neural network, and the number of neurons per layer. Some hyperparameters are shared across algorithms — learning rate shows up in many places. Others belong to one algorithm only — tree depth belongs to decision trees and random forests, while layer count and neuron count belong to neural networks.

Random forest is a strongly explainable algorithm: it does not always match the raw predictive performance of algorithms such as neural networks, but its structure is much easier to interpret. Because of that explainability, random forest hyperparameters such as tree depth and number of trees come up repeatedly as tuning examples throughout this lesson.

Why the ideal hyperparameter value must be searched for

There is no formula that hands over the ideal hyperparameter value. Instead, a practitioner narrows the search range using experience and feasibility.

For example, for a dataset of 1,000 instances, a binary decision tree with a depth of 5 or 6 is a reasonable starting range — going much deeper wastes computational resources without a clear benefit. Similarly, scikit-learn's default learning rate is 0.1, but that default is not always optimal, so a practitioner typically experiments with nearby values such as 0.08, 0.1, 0.17, up to around 0.5.

Because the ideal value is unknown in advance, it has to be searched for. Grid search and randomized search are the two mechanisms this lesson covers for that search.

Grid search: trying every combination

Grid search (implemented in scikit-learn as `GridSearchCV`) evaluates every possible combination of a set of candidate hyperparameter values. The practitioner first lists

candidate values for each hyperparameter to tune; those candidate lists form a grid, with one dimension per hyperparameter.

Consider tuning two random forest hyperparameters: tree depth with candidate values 3, 5, and 7, and number of trees with candidate values 100, 150, and 200. Three values for each of two parameters produces a 3-by-3 grid – 9 total combinations (one combination might be depth 3 with 100 trees, another depth 3 with 150 trees, and so on).

Add a third hyperparameter, minimum instances per leaf, with three candidate values, and the grid becomes a three-dimensional 3-by-3-by-3 grid – 27 total combinations. Grid search is always combined with cross-validation. With 5-fold cross-validation, each of the 27 combinations is trained and tested 5 times (once per fold), so the total number of model-training runs is $27 \times 5 = 135$.

The grid grows exponentially, not linearly, as more hyperparameters or more candidate values are added. With three candidate values per hyperparameter:

Hyperparameters	Combinations
2	$3^2 = 9$
3	$3^3 = 27$
4	$3^4 = 81$
5	$3^5 = 243$

Each additional hyperparameter multiplies the grid size by the number of candidate values, rather than adding a fixed amount. Going from 9 to 27 is a jump of 18, but the underlying operation is a multiplication by 3, not an addition of 9. The same effect appears when the number of candidate values per hyperparameter grows: four hyperparameters, each with five candidate values, produces $5^4 = 625$ combinations. With 5-fold cross-validation, that is $625 \times 5 = 3,125$ model-training runs.

Grid search has a "compulsion": it tries every combination in the grid and then selects the one with the best cross-validation performance. Because it checks every possibility, grid search is guaranteed to find the best combination available in the grid – there is no chance of missing it. That guarantee has a computational cost: on a small dataset of roughly 400 instances, a grid search cell took approximately five minutes to complete in one demonstration run. Real-world datasets, which are far larger, would take substantially longer. A separate illustration used two hyperparameters, each with 10 candidate values, producing a 10-by-10 grid of 100 combinations that grid search would be compelled to try in full.

Watch out for: beyond runtime, this exhaustive cost matters because computing resources such as GPUs and CPUs are expensive and limited. A shared server cannot be occupied for days on a single search when other developers need it, and overheated processors risk shutting a system down. That resource pressure is part of the motivation for a cheaper alternative.

Randomized search: a scaled-down grid search

Randomized search (implemented in scikit-learn as `RandomizedSearchCV`) is described as a scaled-down version of grid search rather than an independent algorithm. It reuses the same grid-formation logic as grid search, but instead of evaluating every combination, it randomly samples a fixed number of combinations, without repeating any combination, and evaluates only those.

The number of random attempts is controlled by a parameter named `n_iter`. For example, from a 100-combination grid, a practitioner might set `n_iter` to 10, meaning only 10 randomly chosen combinations are evaluated with cross-validation, and the best of those 10 is selected.

Random search makes jumps across different regions of the grid rather than clustering in one area. Spreading the attempts across the grid raises the chance that at least one sampled combination lands close to the true best combination. The search only touches a small fraction of the full grid, yet can still perform well. If the search is fortunate, it lands exactly on the best combination; otherwise it typically lands close to it.

Randomized search is not literally "foolproof" the way grid search is — there is a small chance of landing on a poor region of the grid. In practice, spreading attempts keeps that risk low. Historically, randomized search reliably performs close to grid search, both on cross-validation performance and on unseen test data. On unseen data specifically, the combination chosen by randomized search can occasionally *outperform* the exact best combination from grid search. This happens because the "best" grid-search combination can be slightly overfitted to the cross-validation folds, even though cross-validation itself already guards against overfitting to some extent.

`RandomizedSearchCV` has a `random_state` parameter that fixes the random seed. Setting it ensures repeated runs of the same code produce the same randomly selected combinations, which makes results reproducible across runs and across different people running the same code.

The kiwi-basket analogy for choosing `n_iter`: there is no fixed formula for choosing `n_iter`, but this analogy illustrates the reasoning. Imagine a basket with nine pieces of fruit — three kiwis, three oranges, and three apples — and the goal is to pick a kiwi.

- With one attempt, the probability of picking a kiwi is $1/9$, about 11%.
- With two attempts, the probability rises to $2/9$, about 22%.
- With three attempts, the probability rises to $3/9$ (one-third), about 33%.

More attempts increase the odds of success, but too many attempts start to approach the cost of trying every item in the basket, which defeats the purpose of sampling. The same logic applies to `n_iter`: it should be a significant fraction of the total grid size — neither a token handful of attempts nor a number so large it approaches a full grid search.

Watch out for: randomized search does *not* sample "with replacement." Once a combination has been selected, it is not selected again. The terms "with replacement" and "without replacement" are specific to how a random forest builds its individual trees (bootstrap sampling of training rows). They do not describe how randomized search samples hyperparameter combinations — these are two unrelated ideas that are easy to mix up.

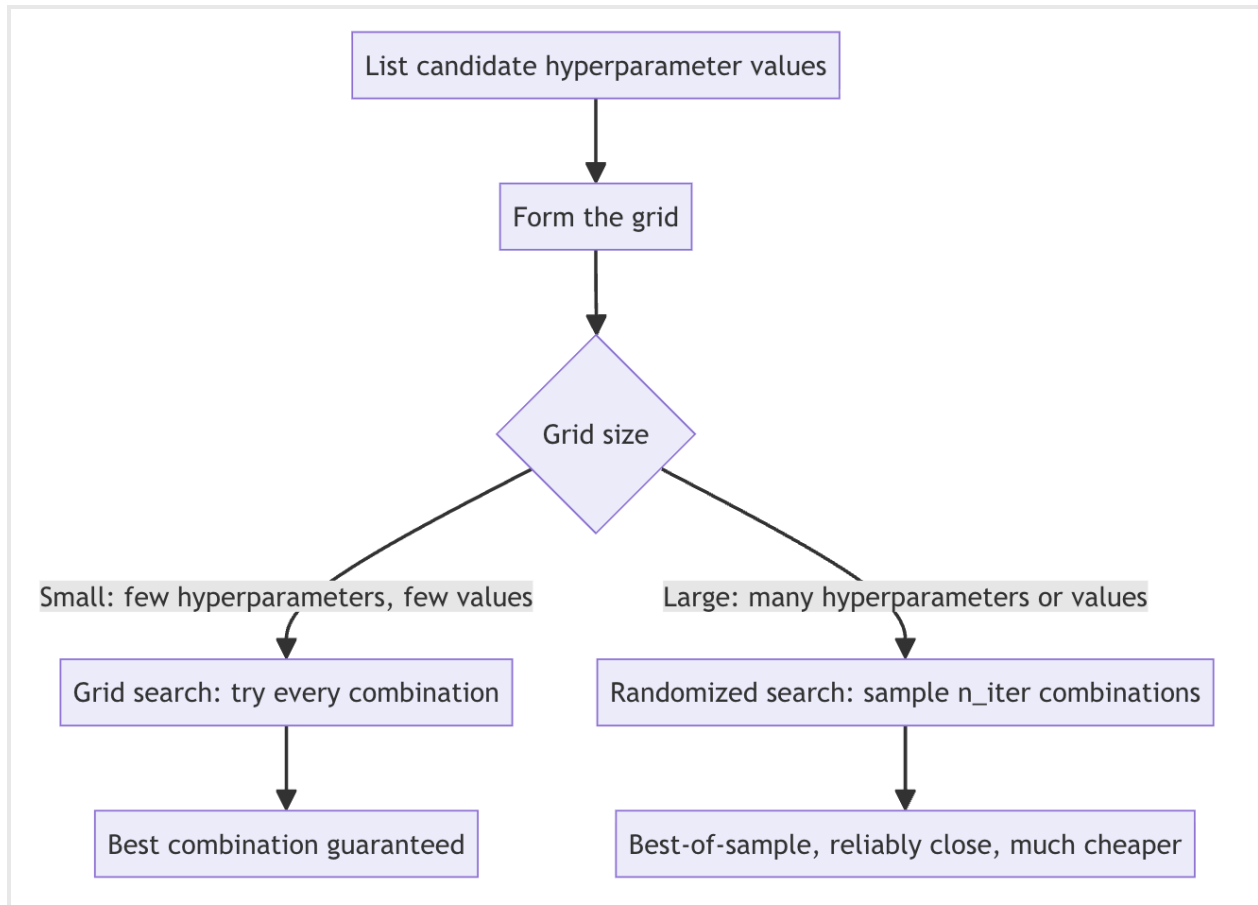
A separate question worth addressing directly: could randomized search end up choosing a combination that performs very poorly, since it only samples a handful of combinations? Because randomized search deliberately jumps across different regions of the grid rather than sticking to one area, it is unlikely to land only on the weakest part of the grid. For example, it avoids clustering only on the left side, only in the center, or only on the right side. That keeps the risk of a very poor outcome low, even though a small risk remains.

Choosing between grid search and randomized search

Both mechanisms are valid hyperparameter optimization strategies; the choice between them depends on how large the grid is.

Aspect	Grid search	Randomized search
Combinations tried	All combinations in the grid	A fixed number (<code>n_iter</code>) of randomly sampled combinations
Guarantee	Always finds the best combination in the grid	Not guaranteed, but reliably close
Computational cost	High, grows exponentially with hyperparameters and candidate values	Much lower, controlled directly by <code>n_iter</code>
Best use case	Few hyperparameters with few candidate values	Many hyperparameters and/or many candidate values

When the number of hyperparameters and candidate values is small, the two approaches cost about the same. For example, tuning a single hyperparameter with three candidate values means $3 \times 5 = 15$ models under 5-fold cross-validation for grid search. A comparably sized randomized search might sample roughly 2 candidate values $\times$ 5 folds = 10 models – the difference (15 versus 10) is too small to matter. When the grid grows large, the difference becomes decisive, and randomized search becomes the practical choice.



Worked example: predicting hospital appointment no-shows

The worked example models a binary classification problem: predicting whether a patient will show up for a scheduled hospital appointment or skip it. Hospitals that allow advance booking want to estimate no-show risk so they can manage patient load.

A synthetic dataset of 400 patient instances was generated with these features:

- Lead time (days): how far in advance the appointment was booked, from a uniform distribution over a 60-day booking window. Booking far in advance is associated with a higher chance of no-show; booking very close to the appointment date suggests the patient is more likely to attend.
- Age: uniform distribution between 18 and 80 years.

- Prior no-show score: how many times the patient has missed past appointments, from a Poisson distribution with $\lambda = 0.5$. This produces mostly small values (zero missed appointments most common, one less common, two rarer still), matching the assumption that most patients usually show up.
- Distance from hospital (km): uniform distribution between 0 and 40 km. Greater distance is a mild deterrent to attending.
- SMS reminder: whether the hospital sent a reminder text, from a binomial distribution giving 0 or 1 with 50% probability each. A reminder is treated as reducing the chance of a no-show.
- Noise feature: a random feature with no real relationship to the outcome, added because real-world data often contains irrelevant features.

The target variable, `no_show`, is binary (1 = did not show up, 0 = showed up). It is built from a weighted linear combination of the features. Lead time and distance carry positive weights; prior no-show score carries a large positive weight since past history is a strong predictor. SMS reminder and age carry negative weights. A bias term is added to produce a raw score, z . The raw score is converted into a probability with the sigmoid function:

```
# convert raw score z into a probability using the sigmoid function
probability_of_no_show = 1 / (1 + np.exp(-z))
```

Each patient's computed probability is compared against a separately generated random threshold value — if the probability exceeds the threshold, the outcome is recorded as a no-show (1), otherwise a show (0). The features and target are combined into a pandas DataFrame with columns for lead time, age, prior no-show record, distance, SMS reminder, noise, and the no-show target.

The data is then split into training and test sets:

```
# split features (X) and target (y) into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

Checking the mean of the target variable confirmed the dataset was balanced: 50% of patients had a no-show outcome and 50% did not.

Running grid search and randomized search on the hospital data

Three random forest hyperparameters were tuned: `n_estimators` (50, 100, 150), `max_depth` (3, 5, 7, or unrestricted), and `min_samples_leaf` (1, 2, 4, 8). That gives a grid of $3 \times 4 \times 4 = 48$ combinations, and with 5-fold cross-validation, grid search trains $48 \times 5 = 240$ models.

```
# imports used across the grid search and randomized search example
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import (
    train_test_split, GridSearchCV, RandomizedSearchCV
)
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# define the hyperparameter grid
param_grid = {
    "n_estimators": [50, 100, 150],
    "max_depth": [3, 5, 7, None],
    "min_samples_leaf": [1, 2, 4, 8],
}

# exhaustive grid search with 5-fold cross-validation
```

```

grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
)
grid.fit(X_train, y_train)

```

Running this cell took approximately five minutes on the demonstration machine, even with a small dataset of 400 instances and only three hyperparameters. The best parameters found were `max_depth=None`, `min_samples_leaf=8`, and `n_estimators=100`, with a best cross-validation score of 0.683 and a test-set accuracy of 0.56. The drop between cross-validation score and test score is larger than would typically be expected on real hospital data; it reflects the fact that this dataset is synthetic rather than collected from an actual hospital.

Randomized search runs on the same grid, but samples only `n_iter=12` combinations:

```

# randomized search samples 12 combinations from the same grid
random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions=param_grid,
    n_iter=12,
    cv=5,
    random_state=42,
)
random_search.fit(X_train, y_train)

```

With `n_iter=12`, randomized search evaluates 12 of the 48 possible combinations, or $12 \times 5 = 60$ models under 5-fold cross-validation, compared with 240 models for the full grid search. Its best cross-validation score was 0.680 — close to grid search's 0.683 — and its test accuracy was 0.57, slightly higher than grid search's 0.56. Randomized search reached comparable performance while training only 60 models instead of 240.

The comparison can be visualized with a bar chart built in Matplotlib, an independent plotting library (scikit-learn also has some of its own plotting utilities for specific outputs like cross-validation curves):

```
# bar chart comparing grid search and randomized search
grid_score = grid.best_score_
random_score = random_search.best_score_

methods = ["Grid Search", "Randomized Search"]
fits = [240, 60]
scores = [grid_score, random_score]

fig, ax = plt.subplots(figsize=(6, 4))
bars = ax.bar(methods, scores, color=["tab:red", "tab:green"])
ax.set_ylim(0, 1)
ax.set_title("Nearly identical quality, a fraction of the fits")
```

The resulting chart shows both methods reaching nearly the same score, while the number of models trained (240 versus 60) differs by a wide margin. Tuning only two or three random forest hyperparameters already produces this gap; random forests commonly have more hyperparameters worth tuning, which widens the gap further in practice.

Applying the approach to real data, and the blind-model sanity check

The mechanics of applying grid search or randomized search to a real dataset are identical to the synthetic example. The only difference is the data source: instead of generating X and y with distributions, a practitioner loads a CSV file into a pandas DataFrame and assigns X and y from its columns. Everything downstream — forming the parameter grid, running `GridSearchCV` or `RandomizedSearchCV`, and evaluating the best estimator — stays the same.

For a balanced binary classification dataset, a hypothetical blind model assigns labels (0 or 1) at random, without learning anything from the data. Because the dataset is balanced, a blind model is expected to be roughly 50% accurate. This baseline is a useful sanity check. If a trained model, whether tuned with grid search or randomized search, performs worse than 50% on a balanced dataset, that signals a real problem — either the data has a flaw or the training process has a flaw. In that situation, the practitioner should investigate the data and process rather than accept the tuned model, and can also increase `n_iter` or fall back to grid search.

Where hyperparameter search fits in the machine learning workflow

Scikit-learn itself is a library, not a workflow: it provides classes and functions for different tasks — for example, a `LinearRegression` class, a `LogisticRegression` class, and a `RandomForestClassifier` class. The workflow itself belongs to the machine learning process, not to the library. The general machine learning workflow is:

1. Prepare the input features (X) and target (y), typically using NumPy and pandas for data handling and cleaning.
2. Split the dataset into training and test portions (for example, with `train_test_split`).
3. Pass the training data to a machine learning algorithm imported from scikit-learn; the algorithm produces a model. That model is ultimately some function — a nested set of if-else rules for a decision tree or random forest, a linear equation for linear regression, a logistic function for logistic regression, or a different function for SVM or neural network algorithms.
4. Use the trained model to predict on the test data.
5. Evaluate performance by computing accuracy or F1 score, typically using scikit-learn's metrics module.

Grid search and randomized search sit inside step 3, as hyperparameter optimization mechanisms for choosing the values used when training the algorithm.

3. Key Takeaways

- Hyperparameters are external settings decided before training, unlike trainable parameters, which the training algorithm learns from the data; since no formula gives the ideal value, both must be searched for.
- Grid search (`GridSearchCV`) tries every combination in a candidate grid and is guaranteed to find the best one, but its cost grows exponentially as hyperparameters or candidate values are added.
- Randomized search (`RandomizedSearchCV`) samples a fixed number (`n_iter`) of combinations without replacement, reliably lands close to the best combination, and can be made reproducible with `random_state`.
- The kiwi-basket analogy shows why `n_iter` should be a meaningful fraction of the grid size — enough attempts to raise the odds of a good pick, without approaching the cost of a full grid search.
- On the hospital no-show worked example, randomized search matched grid search's cross-validation score (0.680 vs. 0.683) and test accuracy (0.57 vs. 0.56) while training only 60 models instead of 240 — and a balanced dataset's blind-model baseline of 50% accuracy is a useful sanity check for any tuned model.

Mental model: think of the hyperparameter grid as a basket of fruit where only a few pieces are "kiwis" (great combinations). Grid search checks every piece in the basket one by one; randomized search grabs a handful of pieces from different parts of the basket, betting that spreading out its picks is enough to land on — or very near — a kiwi.